

Department of Software Engineering

Faculty of Science and Engineering

Database System Lab Manual

CSE 210

GREEN UNIVERSITY OF BANGLADESH

Green University of Bangladesh

Contents

1 Introduction to Database, MySQL, and Managing Databases 2

1.1 Background and software setup 2

1.2 Common data types 3

1.3 Creating and using a database 3

1.4 Creating and inspecting a table 3

1.5 Inserting, browsing, and deleting data 4

2 Implementation of Integrity Constraints in MySQL 5

2.1 Why constraints matter 5

2.2 Primary key, NOT NULL, UNIQUE, DEFAULT and CHECK . 5

2.3 Composite primary key 6

2.4 Foreign-key actions 7

2.5 Controlled tests: expected failures 7

3 Modifying MySQL Tables and Updating Data 8

3.1 ALTER TABLE versus UPDATE 8

3.2 Create a working table 8

3.3 Change table structure 9

3.4 Update and delete records 9

4 Querying and Filtering Data in MySQL 10

4.1 Create a sample employees table 10

4.2 SELECT and DISTINCT 10

4.3 WHERE and comparison operators 11

4.4 Query checklist 11

Final Integrated Lab Task 12

Introduction to Database, MySQL, and Managing Databases

Learning objectives

- Set up a local MySQL-compatible environment using XAMPP and phpMyAdmin.
- Explain fundamental MySQL data types.
- Create, inspect, populate, query and remove database objects.

1.1 Background and software setup

A database is an organized collection of related data. MySQL is a relational database management system; SQL is used to define, manipulate and retrieve data. XAMPP provides a convenient local development environment and phpMyAdmin is a browser-based SQL administration interface.

1. Install XAMPP from `https://www.apachefriends.org/download.html`.
2. Open the XAMPP Control Panel and start the database service (labeled MySQL or MariaDB, depending on distribution). For phpMyAdmin also start Apache.
3. Open `http://localhost/phpmyadmin/` and choose the **SQL** tab.
4. Run the following examples sequentially.

Implementation note

XAMPP often bundles MariaDB rather than Oracle MySQL. The introductory SQL used here is broadly compatible; constraint behavior may vary by database version. For production use, follow your institution's configured platform.

1.2 Common data types

Type	Typical use
INT, BIGINT	Whole numbers and identifiers
VARCHAR(n), CHAR(n)	Variable/fixed length text
TEXT	Long textual content
DECIMAL(p,s)	Exact numeric values such as fees and prices
DATE	Calendar date (YYYY-MM-DD)
DATETIME, TIMESTAMP	Date and time values
BOOLEAN	Logical value (implemented as TINYINT in MySQL)

1.3 Creating and using a database

```
CREATE DATABASE IF NOT EXISTS cse210_lab;
USE cse210_lab;
SHOW DATABASES;
```

1.4 Creating and inspecting a table

```
CREATE TABLE Student (
    StudentID INT,
    FirstName VARCHAR(30),
    LastName VARCHAR(30),
    Address VARCHAR(100),
    City VARCHAR(30)
);
SHOW TABLES;
DESCRIBE Student;
```

1.5 Inserting, browsing, and deleting data

```
INSERT INTO Student (StudentID, FirstName, LastName, Address,
   City)
VALUES
  (1001, 'Utsha', 'Das', 'West Shewrapara', 'Dhaka'),
  (1002, 'Mehedi', 'Hasan', 'Mirpur', 'Dhaka');
SELECT * FROM Student;
```

Implementation note

DROP TABLE removes a table and its data, whereas DELETE FROM removes rows. DROP DATABASE removes the entire database. Execute the commands below only in a disposable practice database, and only after completing the later chapters.

```
-- Execute only when finished with all exercises:
-- DROP TABLE Student;
-- DROP DATABASE cse210_lab;
```

Implementation of Integrity Constraints in MySQL

Learning objectives

- Implement PRIMARY KEY, NOT NULL, UNIQUE, composite keys, and FOREIGN KEY.
- Explore CHECK, DEFAULT, AUTO_INCREMENT and referential actions.
- Interpret and document constraint violations.

2.1 Why constraints matter

Integrity constraints enforce rules about acceptable data. A primary key uniquely identifies each row and cannot contain NULL; a UNIQUE constraint prevents duplicate non-NULL values (MySQL permits multiple NULLs in a UNIQUE column). A foreign key preserves references to a parent table.

2.2 Primary key, NOT NULL, UNIQUE, DEFAULT and CHECK

Run this in a new database to avoid conflicts with Chapter 1:

```
CREATE DATABASE IF NOT EXISTS cse210_constraints;
USE cse210_constraints;
CREATE TABLE Department (
    DeptID INT AUTO_INCREMENT PRIMARY KEY,
    DeptName VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB;
CREATE TABLE Student (
    StudentID INT PRIMARY KEY,
    FullName VARCHAR(80) NOT NULL,
    Email VARCHAR(100) UNIQUE,
```

```
    Age INT NOT NULL,
    Status VARCHAR(15) NOT NULL DEFAULT 'Active',
    DeptID INT,
    CONSTRAINT chk_student_age CHECK (Age >= 16),
    CONSTRAINT fk_student_dept FOREIGN KEY (DeptID)
      REFERENCES Department(DeptID)
      ON DELETE SET NULL ON UPDATE CASCADE
) ENGINE=InnoDB;
INSERT INTO Department (DeptName)
VALUES ('CSE'), ('Software Engineering');
INSERT INTO Student (StudentID, FullName, Email, Age, DeptID)
VALUES (101, 'Rafi Ahmed', 'rafi@example.com', 20, 1),
       (102, 'Nadia Islam', 'nadia@example.com', 21, 2);
DESCRIBE Student;
SELECT * FROM Student;
```

2.3 Composite primary key

A composite primary key combines two or more columns to identify a record uniquely, as in course enrolment.

```
CREATE TABLE Enrolment (
    StudentID INT NOT NULL,
    CourseCode VARCHAR(15) NOT NULL,
    Grade CHAR(2),
    PRIMARY KEY (StudentID, CourseCode),
    FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
) ENGINE=InnoDB;
INSERT INTO Enrolment VALUES
  (101, 'CSE210', 'A'),
  (101, 'CSE211', 'B'),
  (102, 'CSE210', 'A');
```

2.4 Foreign-key actions

Action	Effect on child rows
ON DELETE CASCADE	Deletes matching child rows when the parent row is deleted.
ON DELETE SET NULL	Sets the child foreign-key column to NULL (column must permit NULL).
ON DELETE RESTRICT	Blocks deletion when child rows still reference the parent.
ON UPDATE CASCADE	Updates matching foreign-key values when the parent's referenced key changes.

Implementation note

The supplied manual discusses `SET DEFAULT` as a referential action. Although it appears in the grammar, the InnoDB engine does not support `ON DELETE SET DEFAULT` or `ON UPDATE SET DEFAULT`; therefore, this exercise uses supported actions. CHECK enforcement requires MySQL 8.0.16 or later (or an appropriate MariaDB version).

2.5 Controlled tests: expected failures

Run these one at a time and record error messages. Do not run them as a batch with successful statements.

```
-- Duplicate primary key (must fail):
INSERT INTO Student (StudentID, FullName, Age) VALUES (101, '
   Other', 19);
-- Missing NOT NULL value (must fail):
INSERT INTO Student (StudentID, Age) VALUES (103, 22);
-- Invalid parent reference (must fail):
INSERT INTO Student (StudentID, FullName, Age, DeptID)
VALUES (104, 'Test User', 19, 999);
-- Violates CHECK on supported versions (must fail):
INSERT INTO Student (StudentID, FullName, Age) VALUES (105, '
   Minor', 14);
```

Modifying MySQL Tables and Updating Data

Learning objectives

- Add, rename, modify and remove columns with ALTER TABLE.
- Update selected records safely with WHERE.
- Distinguish schema modification from row modification.

3.1 ALTER TABLE versus UPDATE

`ALTER TABLE` modifies table structure, while `UPDATE` changes stored values. In MySQL, use `MODIFY COLUMN` for a type change and `CHANGE COLUMN` or `RENAME COLUMN` for a name change.

3.2 Create a working table

```
CREATE DATABASE IF NOT EXISTS cse210_modify;
USE cse210_modify;
CREATE TABLE Employee (
    EmpID INT AUTO_INCREMENT PRIMARY KEY,
    FullName VARCHAR(70) NOT NULL,
    Salary DECIMAL(10,2) NOT NULL,
    City VARCHAR(40)
);
INSERT INTO Employee (FullName, Salary, City) VALUES
('Sabbir Rahman', 30000, 'Dhaka'),
('Ananna Rahman', 40000, 'Rajshahi'),
('Jannat Hasan', 45000, 'Dhaka');
```

3.3 Change table structure

```
ALTER TABLE Employee ADD COLUMN Email VARCHAR(100);
ALTER TABLE Employee MODIFY COLUMN City VARCHAR(60);
ALTER TABLE Employee RENAME COLUMN FullName TO EmployeeName;
DESCRIBE Employee;
-- Optional, once Email is no longer needed:
-- ALTER TABLE Employee DROP COLUMN Email;
```

3.4 Update and delete records

```
UPDATE Employee SET Salary = 42000 WHERE EmpID = 2;
UPDATE Employee SET City = 'Gazipur' WHERE EmpID = 1;
SELECT * FROM Employee;
DELETE FROM Employee WHERE EmpID = 3;
SELECT * FROM Employee;
```

Implementation note

Always verify a `WHERE` condition with `SELECT` before `UPDATE` or `DELETE`. Without `WHERE`, a statement may change or delete every row. Some clients enable safe-update mode.

Querying and Filtering Data in MySQL

Learning objectives

- Retrieve whole tables or selected columns with SELECT.
- Eliminate duplicates with DISTINCT.
- Filter rows using WHERE and comparison operators.

4.1 Create a sample employees table

```
CREATE DATABASE IF NOT EXISTS cse210_query;
USE cse210_query;
CREATE TABLE Employees (
    EmpID INT PRIMARY KEY,
    FirstName VARCHAR(40) NOT NULL,
    LastName VARCHAR(40) NOT NULL,
    DOB DATE NOT NULL,
    Gender ENUM('Male','Female'),
    Salary INT NOT NULL,
    EntryDate DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP
);
INSERT INTO Employees (EmpID, FirstName, LastName, DOB, Gender,
    Salary)
VALUES
(1,'Sabbir','Rahman','1998-08-02','Male',30000),
(2,'Sakib','Hasan','1998-08-02','Male',20000),
(3,'Ananna','Rahman','1998-08-02','Female',40000),
(4,'Jannat','Hasan','1998-08-02','Female',45000),
(5,'Sabbir','Hossain','1998-07-02','Male',25000);
```

4.2 SELECT and DISTINCT

```
SELECT * FROM Employees;
SELECT FirstName, LastName, Salary FROM Employees;
SELECT DISTINCT FirstName FROM Employees;
SELECT DISTINCT FirstName, LastName FROM Employees;
```

Implementation note

`DISTINCT FirstName, LastName` removes duplicates of the *entire selected pair*, not duplicate first names alone. It does not delete records from the table.

4.3 WHERE and comparison operators

```
SELECT * FROM Employees WHERE EmpID = 2;
SELECT EmpID, FirstName, Salary FROM Employees
WHERE FirstName = 'Sabbir';
SELECT EmpID, FirstName, Salary FROM Employees
WHERE Salary >= 40000;
SELECT EmpID, FirstName, Salary FROM Employees
WHERE Salary <> 30000;
SELECT EmpID, FirstName, Salary FROM Employees
WHERE Salary < 30000;
SELECT EmpID, FirstName, Salary FROM Employees
WHERE Salary BETWEEN 25000 AND 40000;
```

4.4 Query checklist

Confirm which columns are projected, whether duplicates are removed, which rows satisfy the predicate, and how NULL values affect the result. A `WHERE` predicate retains rows when its condition evaluates to TRUE; SQL NULL handling follows three-valued logic.

Final Integrated Lab Task

Practical Assignment: University Course Registration Database

Goal: Independently design, implement, modify, and query a small database using concepts from all four chapters. **Work individually.** Use MySQL/compatible MariaDB and submit SQL source with output screenshots.

Scenario and minimum schema

Create a database named `UniversityLab_YourID` containing the following tables; choose suitable data types and field lengths.

Table	Required columns
Departments	DeptID, DeptName
Students	StudentID, StudentName, Email, Age, City, DeptID
Courses	CourseID, CourseTitle, Credit, DeptID
Enrollments	StudentID, CourseID, Semester, Grade

Tasks

1. **Database and tables (Chapter 1):** Create the database and all four tables using SQL. Inspect each table using `DESCRIBE`. Insert at least **5 students, 3 departments, 4 courses, and 8 enrollments**. Show all records using `SELECT *`.

2. **Integrity constraints (Chapter 2):** Define primary keys for Departments, Students and Courses; make (StudentID, CourseID, Semester) a **composite primary key** in Enrollments. Apply NOT NULL to essential fields, UNIQUE to Email and DeptName, CHECK to Age (at least 16) and Credit (between 1 and 4), and DEFAULT to at least one suitable column. Establish foreign keys Students → Departments, Courses → Departments, and Enrollments → Students/Courses. Explicitly choose and justify a referential action.

3. **Constraint testing (Chapter 2):** Execute **four separate invalid inserts**: duplicate primary key, duplicate email, invalid foreign-key reference, and a CHECK violation. Capture the error messages and explain each. Do not turn off foreign-key checking.

4. **Schema changes (Chapter 3):** Use `ALTER TABLE` to add a Phone column to Students, change its permitted length, and rename one non-key column. Show table structure before and after. Do not remove a column required in later tasks.

5. **Data changes (Chapter 3):** Use `UPDATE` with `WHERE` to change two students' cities and update a course title. Demonstrate `DELETE` using a safely chosen test row that has no dependent enrolments. Capture the before/after outputs.

6. **Queries (Chapter 4):** Write and show outputs of at least **eight SELECT queries**: (a) all students, (b) selected student columns, (c) DISTINCT city names, (d) a student by primary key, (e) students from a chosen city, (f) students with Age ≥ 20, (g) courses with Credit $\neq 3$, and (h) students with ages within a chosen interval. Use meaningful data so these queries return useful results.

Submission and assessment

Submit: (i) one `.sql` file in execution order; (ii) a PDF lab report containing screenshots of table structures, successful queries, and error messages; (iii) a brief explanation of primary/composite/foreign keys and your referential actions. Label each screenshot with its corresponding SQL query. **Suggested marking scheme (20 marks):** Setup and sample data 4; constraints and violation tests 6; ALTER/UPDATE/DELETE 4; eight queries 4; report quality and explanation 2.

Safety and academic integrity

Work only in your own practice database. Never use destructive statements on shared or production databases. Submit your own SQL, outputs and explanations; copying others' work is not permitted.